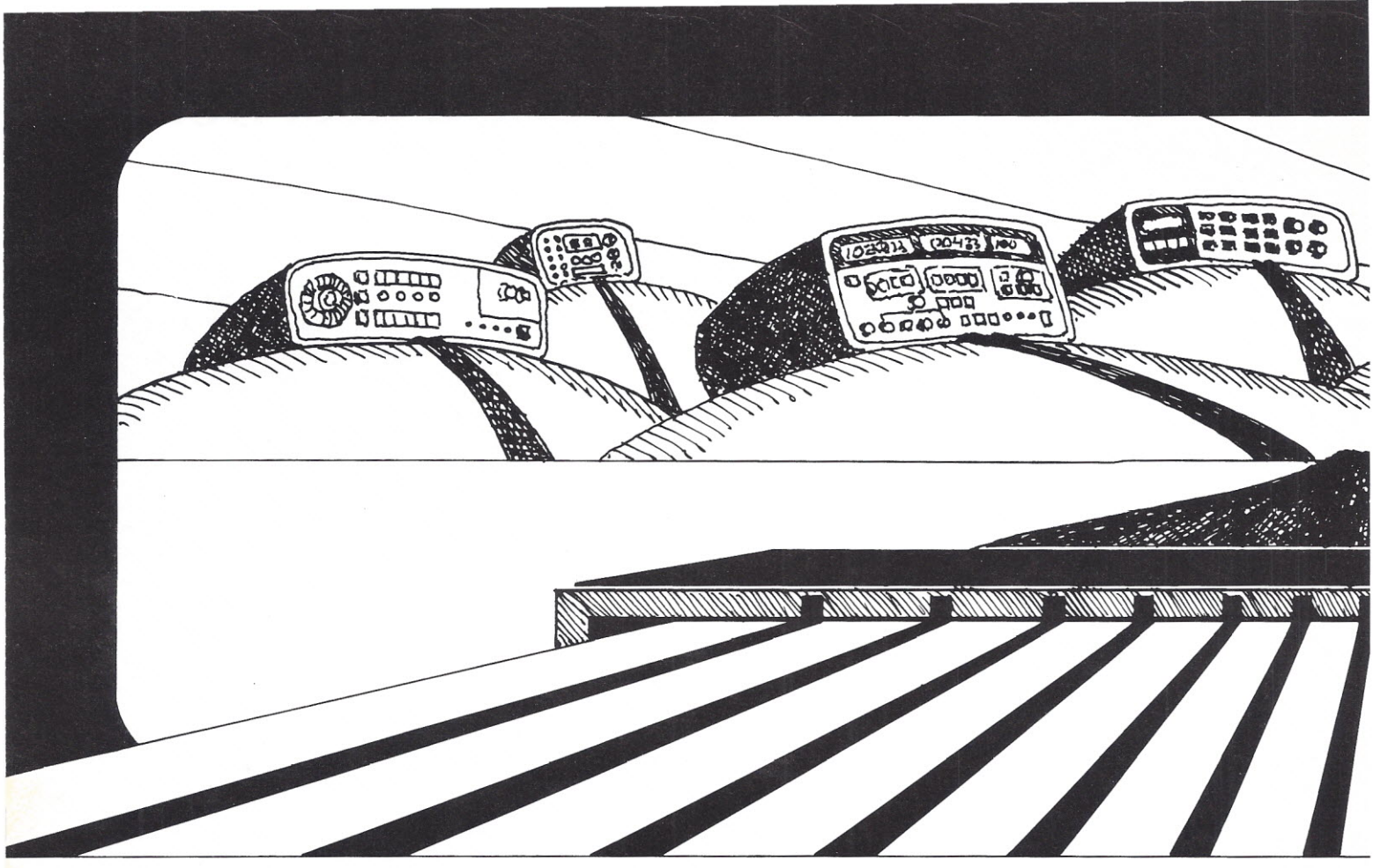


Understanding the IEEE-488 port

Scientific interfacing with the Osborne 1

Wayne Kogachi
Shahriar Kianersi

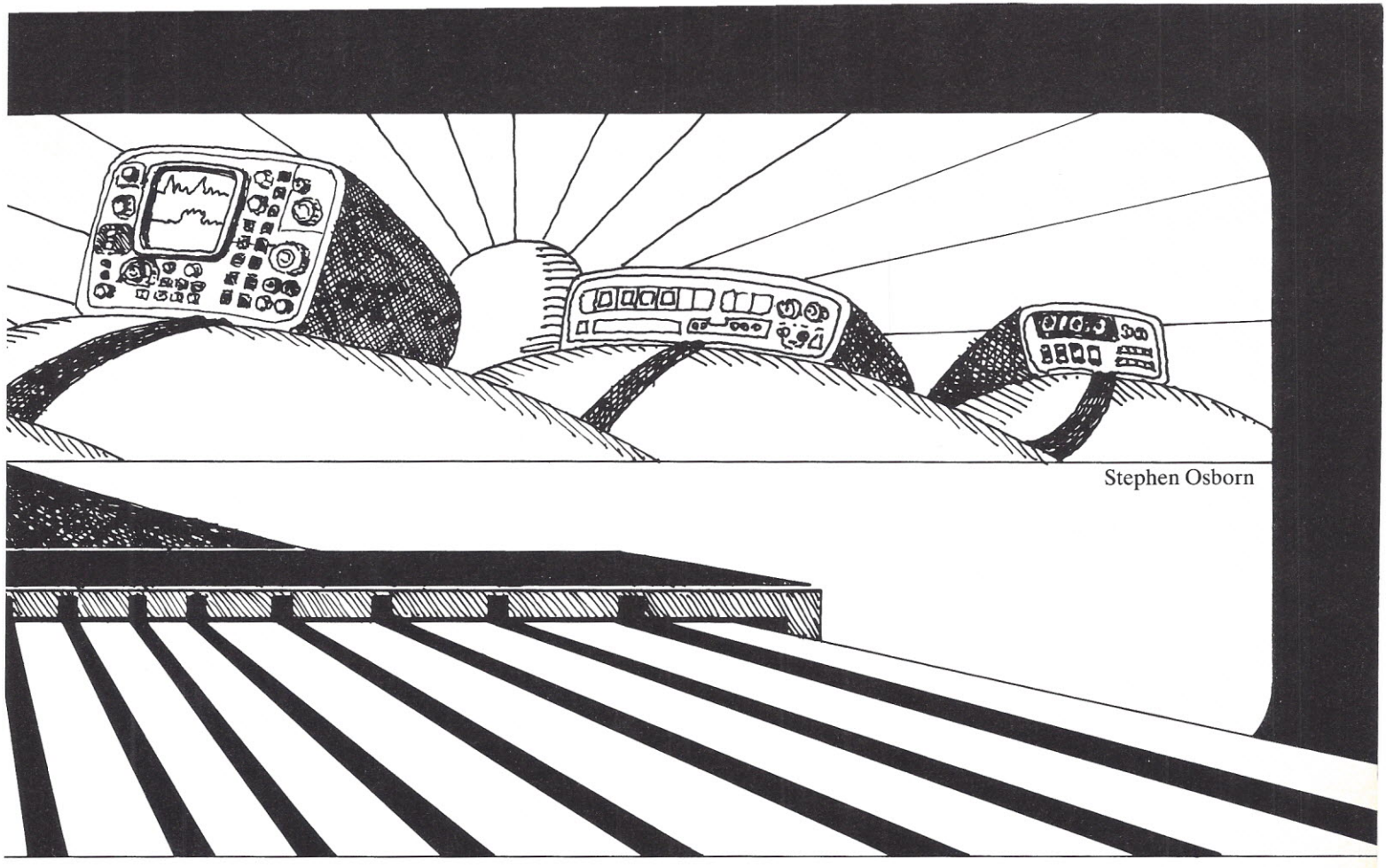
Editor's note—This is an advanced article intended for the engineering and scientific community—or well-versed computer hobbyist. If you're looking for information on intelligent control and interaction with any IEEE interfaced device (e.g. audio analyzer, digital multimeter, quartz thermometer, counter/timer, etc.) this article is for you. Also, read Kelly Smith's joystick article, "Interfacing External Devices," in this issue.

The authors point out that any number of languages can be used to control the IEEE-488, and not just MBASIC as profiled in this article. The principles and concepts are the same.—bb

The dramatic growth of computer technology brought forth the need for certain standards, some of which were in the field of interfaces. Not everybody is an electronics engineer who can build an interface, so standardization takes the worry out of connecting devices to computers.

The IEEE-488 is one of the more widely-used interfaces; it has been published in many languages and is used throughout the world. Over 100 manufacturers use it as the interface for their products, especially in science applications.

The combination of a computer and scientific equipment can be used for such purposes as long-term monitoring, interaction with devices that would otherwise not be possible due to fast response times, monitoring more than one device at a time and handling



Stephen Osborn

massive amounts of data. All this can be accomplished through the IEEE port.

Capabilities of the Osborne IEEE port

Perhaps one of the most valuable features of an IEEE-488 port is its ability to serve more than one device on the data bus. The IEEE-488 port can call up to 15 different devices at one time on the data bus. (From a software point of view, the figure is 29 different devices; however, due to practical hardware limitations only 15 devices are allowed.)

Osborne's IEEE-488 port has eight low-level "drivers," which reside in the ROM and take care of all the IEEE-488 interfacing requirements. These routines are documented in the *User's Guide* in the IEEE-488 implementation chapter. One may invoke any of the routines simply by calling them and passing the proper parameters through preassigned registers. One may also receive messages through assigned registers.

1. Control Out:
   By loading the C register with the proper code you can clear the interface (IFC), and/or set or reset the REN line.

2. Status In:
   Periodically calling this routine will return a value

Table of Talker and Listener

Address Character Talk	Listen	Decimal value
@	SP	ØØ
A	!	Ø1
B	"	Ø2
C	#	Ø3
D	$	Ø4
E	%	Ø5
F	&	Ø6
G	'	Ø7
H	(	Ø8
I	)	Ø9
J	*	1Ø
K	+	11
L	,	12
M	-	13
N	.	14
O	/	15
P	Ø	16
Q	1	17
R	2	18
S	3	19
T	4	2Ø
U	5	21
V	6	22
W	7	23
X	8	24
Y	9	25
Z	:	26
[	;	27
\	<	28
]	=	29
~	>	3Ø

Table 1

in the A register, acknowledging if a device is requesting service.

3. Go To Stand By:
   This will allow other devices on the data bus to take control.

4. Take Control:
   Placing the right code in the C register will let the controller take over the data bus, either synchronously or asynchronously. The error messages, if any, will be returned through the A register.

5. Output Interface Messages:
   The message byte (i.e. the talker or listener address) should be loaded in the C register. Error messages, if any, will return through the A register.

6. Output Device Messages:
   The data byte (for instance, the code to a voltmeter to set it up for particular state) should be loaded in the C register. The end-of-message (or message string) should be acknowledged by placing a code in the B register. Error messages, if any, are returned through the A register.

7. Input Device Messages:
   The data byte (for instance, the data that is measured by a voltmeter) is returned through the A register. The error message, if any, is returned through the L register.

8. Input Parallel Poll:
   The devices that are capable of "parallel poll" will respond by pulling low a preassigned data line so that some kind of code is returned which will correspond to a particular device. If this routine is used, only eight devices can sit on the data bus (one data bit line for each device).

The interface operates at the TTL logic level. After configuring the pin connection for the IEEE standard, one can simply connect the Osborne to any IEEE compatible device. The Osborne's IC chip driver for this interface is a Motorola 6821.

The Osborne 1 is capable of all three device definitions: Talker, Listener and Controller. The way to operate the data bus is to announce the Talker first and then the Listener. Right after this bus management, the Controller will let go of the bus and allow the two devices that now are assigned to the specific duty to take care of the communication (see Table 1 on previous page).

The Controller can announce itself as either the Talker or Listener. It is recommended that the process of the Controller periodically taking over the bus be done "synchronously" rather than "asynchronously." This subject is also covered in the *User's Guide.*

As mentioned earlier, the routines in the ROM are capable of detecting a number of different errors and acknowledging them through proper registers. Some of these errors are:

- Time Out error. A non-fatal error, which simply allows a 100 microsecond grace period of time for the device to respond. Alternately, you can call the device repetitively to allow for slow devices to respond.

- No Device Present error. Either the port is not hooked up to any device, or all the devices sitting on the data bus are turned off.

- Not taking the data after agreeing to take the data.

Before we go any further, it is worth mentioning that if only a printer/plotter is being used on the port there is no need to go through these routines. Using the SETUP program, make sure that the right protocol is chosen (Printers protocols, IEEE-488 compatible) and that the IEEE address port matches the device number. The latter can be simply accomplished with BIOS 1.4. With BIOS 1.3 and earlier versions there is a patch that is provided in the *Technical Tips* section of this issue. Double density computers (1.4 BIOS) have a menu-driven device address patcher.

MBASIC IEEE Routine

The following example is a routine written in MBASIC that communicates with a device on the IEEE port. Part of its routine is to poke a small assembly language routine high in the RAM. In fact, if you have a BIOS 1.3 or earlier version, you should move the CP/M down 1K.

The way these two programs interact with each other is as follows: The MBASIC routine decides which one of the IEEE drivers it wants and what parameter should be passed on through which register. It then puts all that information in locations that are commonly shared by the two routines, so every time that you call the assembly language routine, the control is relinquished to the IEEE drivers. After performing its duty, control is returned back to the MBASIC routine with returned parameters in proper variables.

The "Control Codes" should be in capital letters. Some manufacturers enforce that; otherwise an error message will be returned. One last point: the Osborne 1 address port is zero, so no other device with that address port can sit on the data bus.

Wayne Kogachi is a software engineer with Osborne Computer Corporation. Shahriar Kianersi, also with OCC, is a system support specialist.

IEEE-488 Interface Program

```
100 '
110 '
120 '
130 '   FOR DEMO WITH AN IEEE-488 COMPATIBLE DEVICE
140 '
500 '/* BEGIN MAIN */
510 '
520 GOSUB 3000    'INITIALIZE VARIABLES
530 GOSUB 4000    'POKE ASSEMBLY LANGUAGE ROUTINE
534 '
535 GOSUB 2001    'ASSIGN TALKER LISTENER
536 '
537'    THE STRING "OUTSTR$" CAN CONTAIN THE MESSAGE (OR
538'    MESSAGE STRING) WHICH  WILL BE OUTPUT THROUGH "OUTPUT
539'      DATA MESSAGE" ON TO THE DATA BUS.
540 '
544 OUTSTR$="  "
545 GOSUB 5010       'OUTPUT THE OUTSTR$
546 FOR II=1 TO 10
547 QQ=0             'INITIALIZING
548 GOSUB 2022       'ASSIGN TALKER LISTENER
549 FOR J=1 TO 5
550 GOSUB 5500       'PICK UP THE INCOMING DATA THROUGH "IDM"
551 QQ=VAL(INPUTSTR$)+QQ  'ADD UP 5 SAMPLE FOR AVERAGING
552 NEXT J
553 '
554 QQ=QQ/5 :PRINT"INPUT=     ";QQ
556 '
558 GOSUB 2001            'ASSIGN TALKER LISTENER
559 OUTSTR$="  "          'CHOOSE A NEW STATE FOR THE DEVICE
562 GOSUB 5010            'OUTPUT THE REQUESTED STATE
564 NEXT II
567 '
568 '
598 END    ' /* OF THE MAIN PROGRAM  */
599 '
600 '
1090 '
2000 '/* MAKING THE DEVICE LISTENER */
2001 ' SOME DEVICES REQUIRE SETTING "REN" TRUE BEFORE MAKING THEM
2002 ' THE LISTENER.
2003 POKE CREG%,6:CALL IEEE%(CO%)   'SET "REN" TRUE
2004 POKE CREG%,ASC("?"):CALL IEEE%(OIM%)  'MAKE UNIVERSAL UNLISTEN
2005 POKE CREG%,ASC("@"):CALL IEEE%(OIM%)  'TALKER ADDR (OSBORNE ADDR=00)
2006 POKE CREG%,ASC("<"):CALL IEEE%(OIM%)  'LISTENER ADDR (DEVICE ADDR=28)
2007 RETURN
2008 '
2009 '
2010 '/* END OF MAKING THE DEVICE LISTENER */
2011 '
2020 '/* MAKING THE THE DEVICE TALKER */
2021 '
2022 POKE CREG%,ASC("?"):CALL IEEE%(OIM%)   'MAKE UNIVERSAL UNLISTEN
2023POKE CREG%,ASC("\"):CALLIEEE%(OIM%) 'TALKER ADDR (DEVICE ADDR=28)
2024 POKE CREG%,ASC(" "):CALL IEEE%(OIM%)   'LISTENER ADDR (O-1 ADDR=00)
2025 RETURN
2026 '
2027 '/* END OF MAKING THE DEVICE TALKER */
2028 '
3000 '/* PROCEDURE INITIALIZATION ROUTINE */
3010 '
3020 '    INITIALIZE ALL CONSTANTS USED IN THIS PROGRAM
3030 '
3035 '
3038 '
3040 BEGINADR% = &HE960      'START OF ADDR TO POKE ASSEM LANG ROUTINE
3050 IEEE%     = BEGINADR%   'ADDRESS FOR IEEE CALL
3060 NUMBYTES% = 48          'NUMBER OF BYTES IN ASSEM LANG ROUTINE
3062 EOI%      = 1           'USED FOR END OF MESSAGE
3070 '
3080 CO%    =  0    'CONTROL OUT
3090 SI%    =  1    'STATUS IN
3100 GTS%   =  2    'GO TO STANDBY
3110 TC%    =  3    'TAKE CONTROL
3120 OIM%   =  4    'OUTPUT INTERFACE MESSAGE
3130 ODM%   =  5    'OUTPUT DEVICE MESSAGE
3140 IDM%   =  6    'INPUT DEVICE MESSAGE
3150 PP%    =  7    'PARALLEL POLL
3160 '
3170 INPUTSTR$=STRING$(255," ")
3171 '
3172 RETURN
3180 '
3190 '/* END INITIALIZATION */
3500 '
3600 '
3700 '
4000 '/* PROCEDURE TO POKE ASSEM LANGUAGE ROUTINE INTO HIGH MEMORY */
4010 '
4020 '    THIS PROCEDURE POKES THE DATA INTO HIGH RAM BEGINNING AT
4030 '    LOCATION BEGINADR%
4040 '
4050 FOR I%=0 TO NUMBYTES%      'LOOP FOR POKING DATA
4060     READ A% : HIGHADR%=BEGINADR%+I%
4080     POKE HIGHADR%,A%
4100     IF I%=44 THEN AREG%=HIGHADR%
4110     IF I%=45 THEN BREG%=HIGHADR%
4120     IF I%=46 THEN CREG%=HIGHADR%
4130     IF I%=47 THEN HREG%=HIGHADR%
4140     IF I%=48 THEN LREG%=HIGHADR%
4150 NEXT I%
4160 RETURN
4170 '
4180 '/* END POKE ASSEMBLY LANGUAGE */
4190 '
4200 '
5000 '/* PROCEDURE OUTPUT OUTSTR$ */
5001 '
5002 '    ENTRY - OUTSTR$ ASSIGNED TO OUTPUT STRING
5003 '            MSGTYP% = OIM% OR ODM%
5004 '
5005 '
5010 L%=LEN(OUTSTR$) : IF L%=0 THEN RETURN
5012 I%=0
5030 WHILE I%<L%
5040     POKE CREG%,ASC(RIGHT$(OUTSTR$,L%-I%))
5041     IF I%=L% THEN BREG%,EOI%       ELSE POKE BREG%,0
5060 '   REPEAT
5070        CALL IEEE%(ODM%)
5080     IF PEEK(AREG%)>0 THEN 5060
5085 'PRINT "L=";L%;" I=";I%;" CREG=";HEX$(ASC(RIGHT$(OUTSTR$,L%-I%)))
5090     I%=I%+1
5100 WEND
5110 RETURN
5215 '/* END OUTPUT STRING */
5220 '
5230 '
5500 '/* PROCEDURE TO INPUT FROM IEEE-488 */
5510 '
5520 '    INPUT DATA INTO INPUTSTR$ UNTIL LAST DATA READ
5530 '
5540 INPUTSTR$ = ""
5550 'REPEAT
5560    'REPEAT
5570        CALL IEEE%(IDM%)
5580    IF PEEK(LREG%)>1 THEN 5560
5585    IF PEEK(AREG%)=13 THEN 5560 ELSE IF PEEK(AREG%)=10 THEN RETURN
5590    INPUTSTR$=INPUTSTR$+CHR$(PEEK(AREG%))
5610 IF PEEK(LREG%)<>EOI% THEN 5550
5620 RETURN
5690 '/* END INPUT IEEE-488 */
6000 '
6500 '
7000 '
7500 '
30000 '/* ASSEMBLY LANGUAGE INTERFACE AND REGISTER BUFFER AREA */
30010 '
30020 '    THIS IS DATA FOR AN ASSEMBLY LANGUAGE ROUTINE THAT IS POKED
30030 '    INTO HIGH MEMORY ABOVE CP/M. THIS ROUTINE IS CALLED BY BASIC
30040 '    BY <LINE#> CALL IEEE% (<IEEE FUNCTION#>)
30050 '    BREG% AND CREG% ARE INPUT BUFFERS FOR B AND C Z80 REGISTERS
30060 '    AREG%, HREG%, AND LREG% ARE OUTPUT BUFFERS FOR A,H,L REGISTERS.
30070 '
30080 DATA &H46,&H2A,&H01,&H00,&H2E,&H3F,&H78,&HA7,&HCA,&H74,&HE9,&H16,&H00
30090 DATA &H1E,&H03,&H19,&H05,&HC2,&H6F,&HE9,&H3A,&H8E,&HE9,&H4F,&H3A,&H8D
30100 DATA &HE9,&H47,&HCD,&H8B,&HE9,&H32,&H8C,&HE9,&H7C,&H32,&H8F,&HE9,&H7D
30110 DATA &H32,&H90,&HE9,&HC9,&HE9,&H00,&H00,&H00,&H00,&H00
30120 '
30130 '/* END ASSEMBLY LANGUAGE DATA */
30140 '
```

Lines 30000-30140 contain the assembly language interface routine and buffer area in the form of MBASIC data statements. The source code for this routine is listed below:

```
;PATCH TO MBASIC TO CALL THE IEEE ROUTINES IN BIOS
;ENTRY
;       HL -> NUM CORRESPONDING TO JMP VECTOR FROM BEGINNING OF IEEE BIOS
;             CALLS
;             EXAMPLE: CONTROL OUT = 0
;                      STATUS IN   = 1
;                      ETC.
;       CREG, BREG ARE BYTE LOCATIONS WHICH ARE POKED WITH INPUT VALUES
;       BEFORE A CALL TO THIS ROUTINE.
;       EXAMPLE FOR MBASIC CALL
;             ...
;             IEEE%=                    'ADDR OF ASSEM LANG ROUTINE
;             CONTROLOUT%=0             'IEEE CALL
;             ...
;             AREG%=                    'STORAGE BYTE FOR A REGISTER
;             BREG%=
;             CREG%=
;             ...
;             POKE CREG,1               'INTERFACE CLEAR
;             CALL IEEE(CONTROLOUT%)    'CALL TO IEEE ROUTINE
;             ...
;EXIT
;       AREG, HREG, LREG ARE BYTE LOCATION FILLED WITH A, HL
;       REGISTER VALUES UPON RETURN TO MBASIC

FREESPACE: EQU  0E960H          ;BEGINNING OF 272 BYTES OF UNUSED SPACE

        ORG     FREESPACE       ;BEGINING OF FREE SPACE OF O1 1.4 BIOS

        MOV     B,M             ;FUNCTION# TO B REG
        LHLD    1               ;GET BIOS BEGINNING ADDRESS
        MVI     L,3FH           ;HL->CONTROL OUT BIOS CALL

        MOV     A,B             ;CHECK IF HL POINTS TO CORRECT LOCATION
        ANA     A
        JZ      NEXT

        MVI     D,0             ;BYTES OF CALL ADDRESS
        MVI     E,3

LOOP:   DAD     D               ;NEXT IEEE CALL ADDRESS
        DCR     B
        JNZ     LOOP

NEXT:   ;BIOS IEEE ADDRESS IN HL
        LDA     CREG            ;LOAD REGISTERS FOR INPUT
        MOV     C,A
        LDA     BREG
        MOV     B,A

        CALL    JMPHL           ;CALL IEEE ROUTINE
        STA     AREG            ;SAVE RETURN REGISTERS
        MOV     A,H
        STA     HREG
        MOV     A,L
        STA     LREG
        RET

JMPHL:  PCHL                    ;JUMP TO LOCATION IN HL

;REGISTER BUFFERS

AREG:   DB      0               ;OUTPUT
BREG:   DB      0               ;INPUT
CREG:   DB      0               ;INPUT
HREG:   DB      0               ;OUTPUT
LREG:   DB      0               ;OUTPUT
```